A16 Fragmentos

📕 Fragment

Um Fragment representa o comportamento ou uma parte da interface do usuário em um FragmentActivity. É possível combinar vários fragmentos em uma única atividade para criar uma IU de vários painéis e reutilizar um fragmento em diversas atividades. Você pode imaginar um fragmento como uma seção modular de uma atividade, que tem o próprio ciclo de vida, recebe os próprios eventos de entrada e que pode ser adicionada ou removida durante a execução da atividade (uma espécie de "subatividade" que pode ser reutilizada em diferentes atividades) [1].

Um fragmento deve sempre ser hospedado em uma atividade e o ciclo de vida dele é diretamente impactado pelo ciclo de vida da atividade do host. Por exemplo, quando a atividade é pausada, todos os fragmentos também são e, quando a atividade é destruída, todos os fragmentos também são. No entanto, enquanto uma atividade estiver em execução (estiver no estado do ciclo de vida retomado), é possível processar cada fragmento independentemente, como adicioná-los ou removê-los. Ao realizar tal transação com fragmentos, também é possível adicioná-los a uma pilha de retorno que é gerenciada pela atividade — cada entrada da pilha de retorno na atividade é um registro da transação de fragmento que ocorreu. A pilha de retorno permite que o usuário reverta uma transação de fragmento (navegue para trás), pressionando o botão Voltar [1].

Ao adicionar um fragmento como parte do layout da atividade, ele se encontrará em um ViewGroup dentro da hierarquia de visualizações, e o fragmento definirá o próprio layout da exibição. É possível inserir um fragmento no layout, declarando-o no arquivo de layout da atividade como um elemento , ou no código do aplicativo, adicionando-o a um ViewGroup atual [1].

O Android introduziu os fragmentos no Android 3.0 (API de nível 11) principal-

mente para suportar mais projetos de IU flexíveis e dinâmicos em telas grandes, como em tablets. Como a tela de um tablet é muito maior que a de um celular, há mais espaço para combinar e alternar componentes da IU. Os fragmentos permitem tais projetos sem haver a necessidade de gerenciar alterações complexas na hierarquia de visualizações. Ao dividir o layout de uma atividade em fragmentos, é possível modificar a aparência da atividade em ambiente de execução e preservar essas alterações na pilha de retorno que é gerenciada por essa atividade. Agora estão amplamente disponíveis por meio da biblioteca de suporte a fragmentos [1].

Por exemplo, um aplicativo de notícias pode usar um fragmento para exibir uma lista de artigos na esquerda e outro fragmento para exibir um artigo na direita — ambos os fragmentos aparecem em uma atividade, lado a lado. Cada fragmento tem o próprio conjunto de métodos de callback do ciclo de vida e lida com os próprios eventos de entrada do usuário. Portanto, em vez de usar uma atividade para selecionar um artigo e outra atividade para lê-lo, o usuário pode selecionar um artigo e lê-lo por completo dentro da mesma atividade, como ilustrado no layout do tablet na Fig. 1 [1].

Figura 1: Exemplo de como dois módulos de IU definidos pelos fragmentos podem ser combinados em uma atividade de um projeto para tablet, mas separados em um projeto para celular.

📙 Criação de um fragmento

Para criar um fragmento, é preciso criar uma subclasse de Fragment (ou usar uma subclasse existente dele). A classe Fragment tem um código que é muito parecido com o de uma Activity. Ele contém métodos de callback semelhantes aos de uma atividade, como onCreate(), onStart(), onPause() e onStop(). Na verdade, caso esteja convertendo um aplicativo do Android existente para usar os fragmentos, basta mover o código dos métodos de retorno de chamada da atividade para os respectivos métodos de callback do fragmento [1].

Figura 2: Ciclo de vida de um fragmento.

Figura 2: Ciclo de vida de um fragmento.

Geralmente, deve-se implementar pelo menos os seguintes métodos de ciclo de vida [1].:

- onCreate(): O sistema o chama ao criar o fragmento. Dentro da implementação, deve-se inicializar os componentes essenciais do fragmento que se deseja reter quando o fragmento é pausado ou interrompido e, em seguida, retomado.
- onCreateView(): O sistema chama isso quando é o momento de o fragmento desenhar a interface do usuário pela primeira vez. Para desenhar uma IU para o fragmento, você deve retornar uma View deste método, que é a raiz do layout do fragmento. É possível retornar como nulo se o fragmento não fornecer uma IU.
- onPause(): O sistema chama esse método como o primeiro indício de que o usuário está saindo do fragmento (embora não seja sempre uma indicação de que o fragmento está sendo destruído). É quando geralmente se deve confirmar qualquer alteração que deva persistir além da sessão do usuário atual (porque o usuário pode não retornar).

A maioria dos aplicativos deve implementar pelo menos três destes métodos para cada fragmento, mas há vários outros métodos de callback que você deve usar para lidar com diversos estágios do ciclo de vida do fragmento. Todos os métodos de callback do ciclo de vida são abordados com mais detalhes na seção Processamento do ciclo de vida dos fragmentos [1].

```
class ExamploFragment : Fragment() {

    override fun onCreateView(inflater: LayoutInflater,
                              container: ViewGroup?,
                              savedInstanceState: Bundle?): View {
        // Carrega o layout do fragmento
        return inflater.inflate(R.layout.example_fragment, container,
false)
    }
```

```
}
```

📗 Adição de um fragmento a uma atividade

Geralmente, um fragmento contribui com a atividade do host com uma parte da IU, que é incorporada como parte da hierarquia de visualizações geral da atividade. Há duas formas de adicionar um fragmento ao layout da atividade:

- Declarar o fragmento dentro do arquivo de layout da atividade: neste caso, é possível especificar as propriedades do layout para o fragmento como se fosse uma visualização. Por exemplo, veja o arquivo de layout para uma atividade com dois fragmentos:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/an-
droid"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <fragment android:name="com.example.news.ArticleListFragment"
            android:id="@+id/list"
            android:layout_weight="1"
            android:layout_width="0dp"
            android:layout_height="match_parent" />
    <fragment android:name="com.example.news.ArticleReaderFragment"
            android:id="@+id/viewer"
            android:layout_weight="2"
            android:layout_width="0dp"
            android:layout_height="match_parent" />
</LinearLayout>
```

- Ou adicionar programaticamente o fragmento a um ViewGroup existente. A qualquer momento, enquanto a atividade está em execução, é possível adicionar fragmentos ao layout da atividade. Basta especificar um ViewGroup no qual posicionar o fragmento.

```
val fragment = ExampleFragment() // Instanciação do fragmento

// Carrega o fragmento no ViewGroup com id container
val fragmentTransaction = supportFragmentManager.beginTransaction()
fragmentTransaction.add(R.id.container, fragment)
fragmentTransaction.commit()
```

📘 DataBinding

Para utilizar as facilidades do `DataBinding` com fragmentos, o processo é um pouco diferente do processo com as atividades.

```
class ExamploFragment : Fragment() {

        override fun onCreateView(inflater: LayoutInflater,
                                  container: ViewGroup?,
                                  savedInstanceState: Bundle?): View {
                // Cria a view usando data binding
        val binding = ExamploFragmentBinding.inflate(inflater, contai-
ner, false)

                ...

                return binding.root
    }
}
```

📚 Referência bibliográficas

[1] Fragments. **Android Developers**. Acessado em 14/01/2021.